Take Gifts From the Richest Pile(easy)

Problem Statement

You're presented with several piles of gifts, with each pile containing a certain number of gifts. Every second, you'll engage in the following activity:

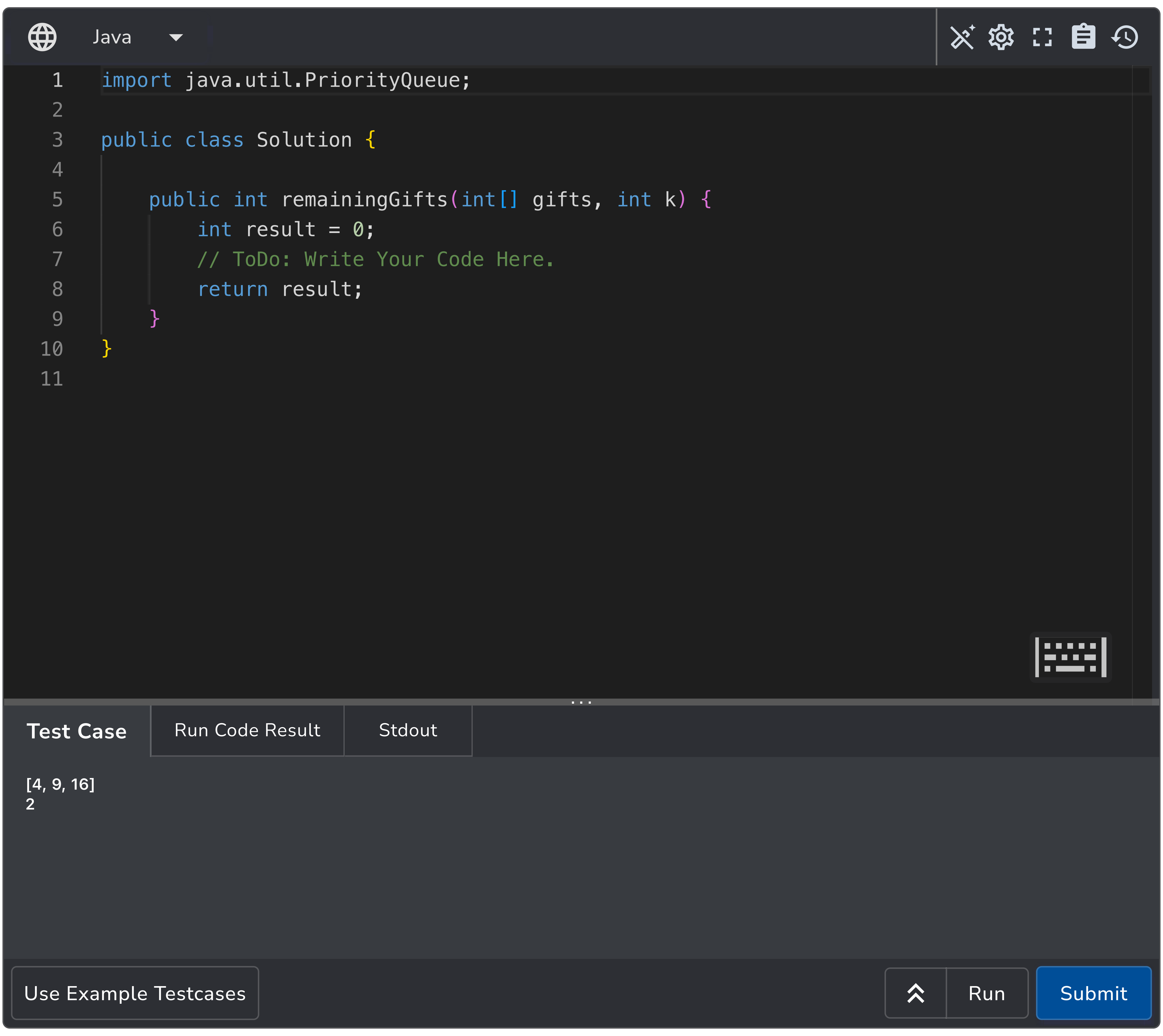
- Pick the pile that contains the highest number of gifts. If multiple piles share this distinction, you can select any of them.
- Compute the square root of the number of gifts in the selected pile, and then leave behind that many gifts (rounded down). Take all the other gifts from this pile.
- You'll do this for "k" seconds. The objective is to find out how many gifts would still remain after these "k" seconds.

Examples

1. - **Input**: gifts = [4, 9, 16], k = 2
   - **Expected Output**: 11
   - **Justification**:
     1. Take from third pile (16 gifts): leave ($\sqrt{16}$) = 4 gifts, take 12. Remaining gifts = [4, 9, 4]
     2. Take from second pile (9 gifts): leave ($\sqrt{9}$) = 3 gifts, take 6. Remaining gifts = [4, 3, 4]
2. - **Input**: gifts = [1, 2, 3], k = 1
   - **Expected Output**: 4
   - **Justification**:
     1. Take from third pile (3 gifts): leave ($\sqrt{3}$) = 1 gift (rounded down), take 2. Remaining gifts = [1, 2, 1]
3. - **Input**: gifts = [25, 36, 49], k = 3
   - **Expected Output**: 18
   - **Justification**:
     1. Take from third pile (49 gifts): leave ($\sqrt{49}$) = 7 gifts, take 42. Remaining gifts = [25, 36, 7]
     2. Take from second pile (36 gifts): leave ($\sqrt{36}$) = 6 gifts, take 30. Remaining gifts = [25, 6, 7]
     3. Take from first pile (25 gifts): leave ($\sqrt{25}$) = 5 gifts, take 20. Remaining gifts = [5, 6, 7]

Try it yourself

Try solving this question here:



```java
import java.util.PriorityQueue;

public class Solution {

    public int remainingGifts(int[] gifts, int k) {
        int result = 0;
        // ToDo: Write Your Code Here.
        return result;
    }
}

```

```
[4, 9, 16]
2
```

← Previous
Introduction To Heap
Next →
Solution: Take Gifts From The Richest Pile(Easy)
☐ Mark as Completed